



TEORÍA DE ALGORITMOS (TB024/75.29)

Trabajo Práctico N° 2

**Redes de flujo, Clases de
Complejidad y Fuerza Bruta**

Grupo 3SAT

Profesor:
Ing. Victor Podberezky

Versión 2 (Reentrega N° 1)
8 de junio de 2026

Índice

1. Introducción	2
2. Desarrollo del trabajo	2
2.1. La separación del Dojo	2
2.1.1. Explicación del problema	3
2.1.2. Propuesta de solución por red de flujos	3
2.1.3. Modelado de Nodos (V)	4
2.1.4. Modelado de Aristas (E) y Capacidades (c)	4
2.1.5. Justificación del enfoque por Corte Mínimo	4
2.1.6. Pseudocódigo	5
2.1.7. Análisis de la solución	6
2.1.8. Ejemplo gráfico de ejecución de la solución	9
2.2. El despacho de los productos	11
2.2.1. Explicación del problema	12
2.2.2. Búsqueda exhaustiva sin poda	12
2.2.3. Propuesta	12
2.2.4. Explicación del funcionamiento	13
2.2.5. Función costo	13
2.2.6. Función límite	13
2.2.7. Complejidades	14
2.2.8. Ejemplo de ejecución	14
2.2.9. Pseudocódigo	15
2.3. Los centros de atención	16
2.3.1. Reducción polinomial a Set Cover	16
2.4. Versión de decisión	18
2.4.1. Otras cuestiones respecto a las clases de complejidad	19
3. Correcciones en la segunda versión	20
4. Conclusión	20
5. Referencias	21

1. Introducción

El estudio avanzado de algoritmos permite no solo resolver problemas de manera eficiente, sino también comprender las limitaciones intrínsecas de la computación y la estructura subyacente de problemas complejos de optimización. Mientras que las técnicas de diseño clásicas proporcionan herramientas para construir soluciones, el análisis de flujos en redes y la clasificación de problemas según su complejidad permiten determinar la viabilidad de dichas soluciones en entornos reales.

En el presente informe, se profundizará en el estudio de tres ejes fundamentales de la algoritmia moderna: la modelización mediante redes de flujo, la implementación de estrategias de búsqueda exhaustiva y el marco teórico de las clases de complejidad. En la primera parte de este trabajo, se abordará la teoría de Redes de Flujo, una herramienta esencial para resolver problemas de transporte, logística y asignación de recursos. Se analizará cómo modelar un sistema de capacidades para determinar el flujo máximo posible entre nodos, aplicando conceptos como el teorema de Max-Flow Min-Cut.

Posteriormente, se explorará la técnica de Fuerza Bruta, evaluando su utilidad como punto de partida para la resolución de problemas combinatorios y discutiendo el costo computacional que implica la exploración completa del espacio de soluciones.

Finalmente, se presentará un análisis sobre las Clases de Complejidad, estableciendo las distinciones fundamentales entre problemas de las clases P , NP y NP -completos. Este apartado permitirá comprender la frontera entre lo tratable y lo intratable, brindando los criterios necesarios para identificar cuándo un problema exige una solución heurística o aproximada frente a la imposibilidad de hallar una solución exacta en tiempo polinomial.

2. Desarrollo del trabajo

2.1. La separación del Dojo

Las redes de flujo permiten modelar una gran variedad de problemas. Es posible representar cada arista dirigida como un conducto con capacidad limitada para transportar un fluido, y cada vértice como una intersección de dichos conductos. En consecuencia, por la ley de conservación, la cantidad de fluido que sale de cada intersección debe ser igual a la que ingresa. Gracias a

esta relación de equilibrio, es posible identificar características que permiten resolver problemas complejos mediante una reducción polinomial a una red de flujo.

El conocimiento sobre el comportamiento del flujo, el valor máximo que puede alcanzar la red y la identificación del “cuello de botella”, determinado por el corte que minimiza la capacidad entre la fuente y el sumidero, es lo que permite adaptar este modelo a problemas que, a simple vista, no parecen tener una estructura de red. De esta forma, se logran soluciones algorítmicas eficientes para desafíos de optimización y particionamiento.

2.1.1. Explicación del problema

Un dojo tiene “ n ” miembros, dos de ellos son profesores, cada uno de los miembros del dojo se relacionó al menos una vez con alguno de los profesores. Dado un grupo de relaciones R_A , cuyos elementos son tuplas del tipo $[(m_i, m_j), r_{ij}]$ donde m_i y m_j son miembros del dojo tal que $m_i \neq m_j$ y r_{ij} es un entero positivo representativo de las veces que se relacionó el miembro m_i con m_j , se busca dividir a los miembros del dojo entre los dos profesores de forma que la cantidad de veces que se relacionaron dos miembros de un grupo sea la máxima posible.

Supuestos del problema: Se supone que cada miembro del dojo se relacionó al menos una vez con alguno de los profesores, y que en el grupo de relaciones no pueden haber relaciones negativas. Además se supone que el primer profesor p_1 tiene número de miembro 1 y el profesor p_2 tiene numero de miembro “ n ”.

2.1.2. Propuesta de solución por red de flujos

Para resolver el problema de la separación del Dojo asegurando que la suma de las intensidades de las relaciones rotas sea mínima, modelamos la situación mediante una red de flujo $G = (V, E)$ con capacidades asociadas a las aristas. El objetivo principal consiste en hallar un corte mínimo que separe a los dos profesores rivales (p_1 y p_2).

A continuación, se detalla el diseño y la estructura formal de la red de flujo propuesta:

2.1.3. Modelado de Nodos (V)

El conjunto de vértices V de la red representa a la totalidad de los integrantes del dojo. Cada profesor o maestro se corresponde con un nodo único dentro del grafo. Adicionalmente, se identifican de manera fija dos nodos especiales que actúan como terminales de la red:

- **Fuente (s):** Corresponde al Profesor 1 (p_1), quien encabezará el primer subgrupo.
- **Sumidero (t):** Corresponde al Profesor 2 (p_2), quien encabezará el segundo subgrupo enfrentado.

2.1.4. Modelado de Aristas (E) y Capacidades (c)

Dado que las relaciones de afinidad o tensión entre los miembros del dojo son simétricas y recíprocas (es decir, si el profesor u tiene un vínculo de peso w con el profesor v , el profesor v posee inherentemente el mismo grado de vínculo con u), **las relaciones deben modelarse de forma bidireccional**.

Para representar correctamente esta naturaleza no dirigida dentro de una red de flujo (la cual es por definición dirigida), cada relación de peso w provista en el enunciado entre un par de nodos u y v se traduce en **dos aristas dirigidas simétricas** en el grafo:

- Una arista dirigida (u, v) con capacidad inicial $c(u, v) = w$.
- Una arista dirigida (v, u) con capacidad inicial $c(v, u) = w$.

En caso de existir múltiples especificaciones de relaciones o vínculos superpuestos entre el mismo par de nodos en el conjunto de datos de entrada, las capacidades correspondientes se acumulan de forma aditiva, garantizando la consistencia del modelo:

$$c(u, v) = c(v, u) = \sum w_{uv}$$

2.1.5. Justificación del enfoque por Corte Mínimo

Al aplicar el teorema de *Max-Flow Min-Cut* (Flujo Máximo - Corte Mínimo), sabemos que el valor del flujo máximo que puede transportarse desde la fuente s hasta el sumidero t es exactamente igual a la capacidad del corte

mínimo que separa a ambos nodos en dos subconjuntos disjuntos de vértices, S (donde $s \in S$) y T (donde $t \in T$).

Las aristas que atraviesan este corte (es decir, aquellas aristas dirigidas que van desde el subconjunto S hacia el subconjunto T y cuyas capacidades residuales quedan completamente saturadas en 0 al finalizar la ejecución del algoritmo) representan con precisión matemática los vínculos de afinidad que deben disolverse. De este modo, la partición de la red en los componentes disjuntos S y T determina de manera óptima la conformación de los dos nuevos dojos independientes, minimizando estrictamente el costo total de las relaciones rotas.

2.1.6. Pseudocódigo

Una posible implementación del código solución de esta propuesta puede ser el siguiente:

```
function definirGruposDojo(miembros V, relaciones R, p1, p2):
    G = Grafo nuevo

    for cada relacion (u, v, peso) in R:
        G.agregarVertice(u)
        G.agregarVertice(v)

        G.capacidad[u][v] = G.capacidad[u].get(v, 0) + peso
        G.capacidad[v][u] = G.capacidad[v].get(u, 0) + peso

    s = p1
    t = p2

    # Obtenemos el grafo residual tras aplicar Edmonds-Karp.
    G_res = EdmondsKarp(G, s, t)

    grupo_p1 = {s}
    cola = [s]

    while cola no esta vacia:
        u = cola.desencolar()
        for cada vecino v de u en G_res:
            # Solo avanzamos si hay capacidad residual > 0.
            if v no esta en grupo_p1 y G_res.capacidad[u][v]
> 0:
                grupo_p1.agregar(v)
                cola.encolar(v)
```

```

grupo_p2 = G.V - grupo_p1

return grupo_p1, grupo_p2

```

Listing 1: Pseudocódigo de la solución

Donde la función `FlujoMaximo` está determinada por el siguiente Pseudocódigo:

```

function EdmondsKarp(Grafo G, s, t):
    # Inicialmente el grafo residual es igual al original.
    G_res = copiar(G)
    flujo_total = 0

    camino = BFS_camino_aumento(G_res, s, t)

    while camino existe:
        flujo_camino = minimo_cuello_botella(camino)

        for cada arista (u, v) en camino:
            G_res.capacidad[u][v] -= flujo_camino # Restamos
            flujo a la arista directa.
            G_res.capacidad[v][u] += flujo_camino # Sumamos
            flujo a la arista inversa.

        flujo_total += flujo_camino
        camino = BFS_camino_aumento(G_res, s, t)

    return G_res

```

Listing 2: Pseudocódigo de `FlujoMaximo`

2.1.7. Análisis de la solución

A partir del pseudocódigo presentado anterior se puede realizar un análisis en profundidad de la solución propuesta, comenzando por un análisis de uso de recursos de tiempo y espacio, para luego terminar de definir detalles que pueden haber quedado sin analizar.

Análisis de eficiencia espacial: En cuestiones de uso de espacio, este algoritmo ocupa distintos ordenes de complejidad según como se implemente el grafo a utilizarse. En caso de usar una lista de adyacencia, la complejidad

espacial de guardar el grafo de relaciones conllevaría $O(V + E)$, mientras que una matriz de adyacencias utiliza $O(V^2)$. Dado que en este trabajo se implementaron las soluciones algorítmicas en el lenguaje de programación Python, se utilizó la lista de adyacencias¹ y por lo tanto, en consecuencia a este hecho, la complejidad espacial del algoritmo será igual a $O(V + E)$. Pues guardar todos los integrantes requiere n vértices o también $O(V)$ y guardar cada una de las aristas requiere m aristas para una dirección de las relaciones y m aristas para la otra, lo que es lo mismo que $O(2E)$. Esto resulta en la ecuación de complejidad espacial:

$$O(V) + O(2E) = O(V + 2E) = O(V + E)$$

Cabe aclararse adicionalmente, que la operatoria de búsqueda de flujo máximo requiere $O(V)$ espacio adicional, que por sumarse a la complejidad mencionada anteriormente no afecta al requerimiento espacial final ya que $O(V) \subset O(V + E)$.

Análisis de eficiencia temporal: Respecto al tiempo de ejecución del algoritmo, construir la red de relaciones lleva $O(V + E)$ en ordenes de tiempo e identificar los distintos subconjuntos también requieren $O(V + E)$. Dado que se eligió el algoritmo de Edmonds-Karp, este tiene una complejidad de $O(V + E^2)$ dado que iterando por cada arista ($O(E)$) se busca un camino de aumento ($O(V + E)$) lo que resulta en la complejidad dada. Adicionalmente, contabilizar cuantos miembros contienen cada subconjuntos puede conllevar como mucho una complejidad de $O(V)$.

Ya que $O(V) \subset O(V + E)$ y $O(V + E) \subset O(V + E^2)$, la complejidad resulta:

$$2O(V + E) + O(V + E^2) + O(V) = O(V + E^2) \quad (1)$$

Otros detalles de la solución: En esta subsección del análisis de la propuesta algorítmica se busca responder a las preguntas “¿*Realmente la propuesta predicará la división final?*” y “¿*Puede una misma instancia tener diferentes soluciones?* ¿*Que implicación tiene elegir una solución por encima de otra?*”.

Respecto a la primer pregunta, esta solución no predicará la división final entre los miembros, sino que esta solución presenta un modelo de estimación simplificada. Esto está dado porque el corte mínimo asume que el sistema busca el estado de menor fricción, es decir la separación del grupo de miembros del dojo que mantenga los vínculos más fuertes. Adicionalmente, dado

¹En este caso sería más adecuado decir “Diccionario de adyacencias”.

que es un modelo estimativo, puede ocurrir en la realidad que por fuera de toda estimación dada por esta solución, un alumno elija irse con un profesor en vez del estimado. Esta variación además podría generar un efecto cascada de cambio de profesor de alumnos de relación fuerte con el miembro que eligió al otro profesor.

Otra característica de esta solución es el hecho de que pueden haber múltiples opciones de corte en una misma instancia. En una red de flujo, el flujo máximo toma un único valor, pero el corte mínimo de capacidades de dicha red no tiene porque ser uno solo. Tomando el caso donde hay un solo miembro además de ambos profesores y que dicho miembro interactuó una misma cantidad de veces con ambos profesores, ocurre el caso donde la red de flujo tendrá un flujo máximo igual al número de veces que haya interactuado el alumno con los profesores pero el corte que minimiza las capacidades puede realizarse desde la fuente o desde el sumidero, por lo que dará distintas soluciones según como se haya determinado la operación de corte.

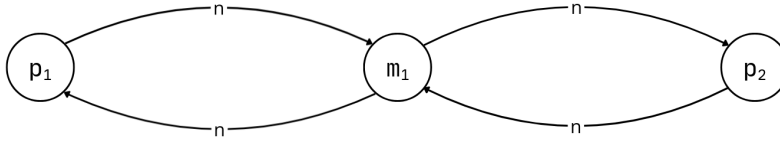


Figura 1: Caso borde donde se pone en duda donde se realizará el corte mínimo

Este problema de incertidumbre es equivalente para cualquier cantidad de miembros, siempre que se cumpla que ambos profesores hayan interactuado la misma cantidad de veces con ellos y no hayan interacciones entre miembros. Consecuentemente, si bien el Teorema de Flujo Máximo-Corte Mínimo ofrece una solución óptima en términos de capacidad, la existencia de múltiples cortes mínimos sugiere que ciertos miembros se encuentran en una posición de equilibrio social. La elección de la fuente (s_1) para iniciar la exploración del grafo residual inclina la asignación de estos miembros en disputa hacia el primer profesor.

2.1.8. Ejemplo gráfico de ejecución de la solución

En esta sección se mostrará la solución de una instancia del problema según el algoritmo mostrado anteriormente. Dada la instancia siguiente instancia:

$[(1, 2], 4), ([1, 3], 4), ([1, 5], 1), ([2, 5], 1), ([5, 7], 5), ([3, 4], 4), ([5, 6], 5),$
 $([6, 7], 5), ([4, 5], 1)]$

Con el profesor 1 representado por el número 1 y el profesor 2 representado por el número 7. Se construye una red de flujo a partir de ella, donde los nodos fuente y sumidero son el profesor 1 y el profesor 2 respectivamente, los nodos intermedios son los miembros del dojo y las aristas dirigidas y ponderadas son las veces que interactuaron dos miembros. La instancia elegida para este ejemplo conforma la red mostrada en la Figura 4.

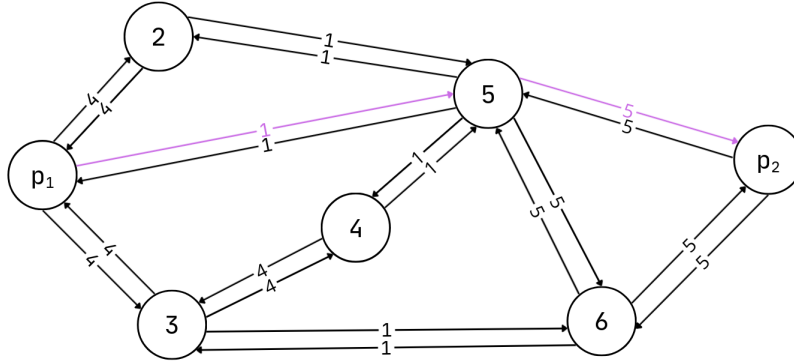


Figura 2: Red conformada por la instancia dada.

En una primera iteración, el algoritmo busca el camino más corto entre p_1 y p_2 (marcada en violeta) y busca el cuello de botella de dicho camino para poder saturar sus aristas, tanto de una dirección como de otra. En este caso como el cuello de botella es 1, se pasa una unidad de flujo, restando en una dirección y sumando en la otra.

Esta operatoria se repite hasta que no existan más caminos que conecten p_1 con p_2 . Cuando esto ocurra, el flujo máximo de la red será la cantidad de unidades de flujo acumuladas tras la saturación de aristas.

En este ejemplo se encuentra en una segunda iteración el camino $p_1 \rightarrow 2 \rightarrow 5 \rightarrow p_2$ cuyo cuello de botella es la arista $(2 \rightarrow 5)$ y su complemento en la otra dirección, con una capacidad de 1. En una tercera operación, se satura la arista $(3 \rightarrow 6)$ de capacidad 1, correspondiente al camino $p_1 \rightarrow 3 \rightarrow 6 \rightarrow p_2$.

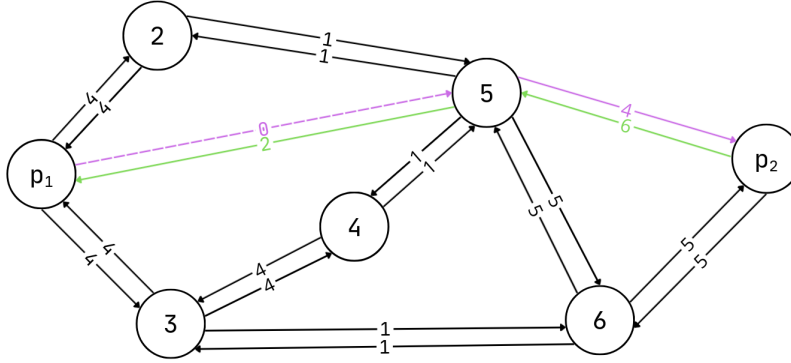


Figura 3: Red con 1 unidad de flujo acumulada.

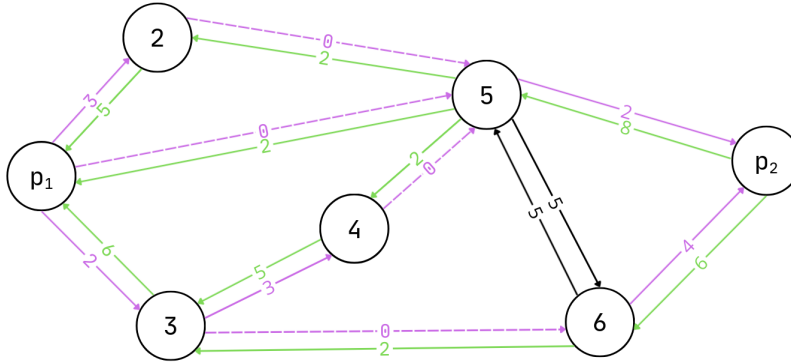


Figura 4: Red sin caminos $p_1 \rightarrow p_2$, con 4 unidades de flujo acumulado

Y por último se encuentra el camino más corto $p_1 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow p_2$ que encuentra el cuello de botella en $(3 \rightarrow 4)$ de capacidad 1.

Entonces a partir de estos pasos realizados, el corte mínimo que particiona a la red de forma que se minimice la cantidad de relaciones perdidas tras la separación será el corte que atraviese aquellas aristas que tras saturarse resultan tener capacidad 0.

Una vez logrado saturar las aristas de la red, se pueden obtener ambos grupos separando aquellos nodos que aún son alcanzables por p_1 , en este ejemplo $\{2, 3, 4\}$, de los que no son alcanzables $\{5, 6\}$

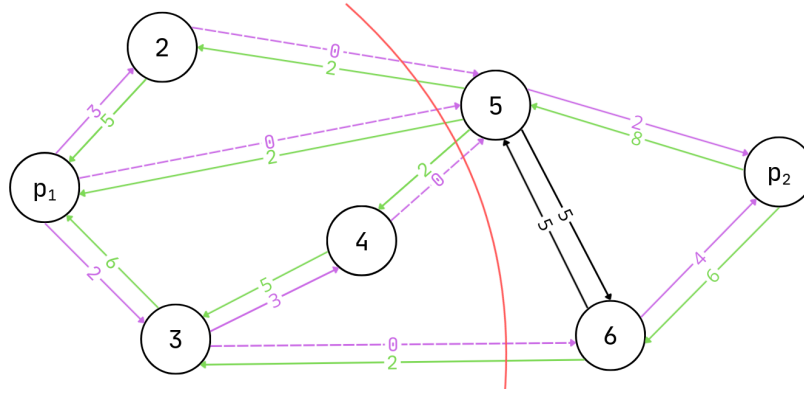


Figura 5: Corte mínimo de la instancia.

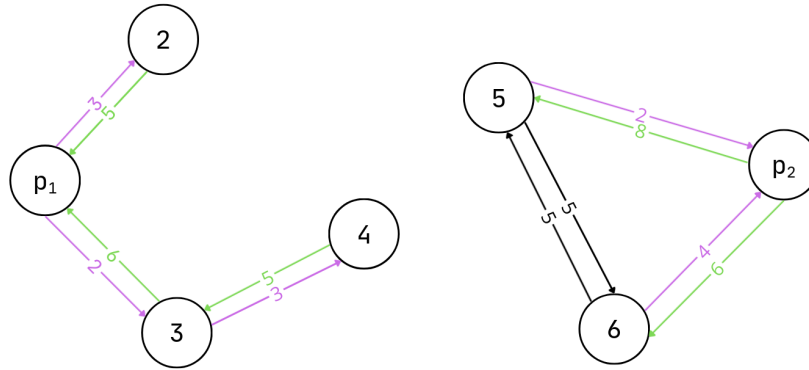


Figura 6: Grupos resultado de la instancia

2.2. El despacho de los productos

En muchos problemas de logística y optimización de recursos no solo importa maximizar una ganancia, sino también minimizar las pérdidas asociadas al desperdicio de capacidad o recursos no utilizados. Este tipo de situaciones aparece frecuentemente en problemas de empaquetado, transporte y asignación de tareas, donde encontrar una solución óptima requiere analizar múltiples configuraciones posibles.

En esta sección se resolverá un problema de optimización mediante la técnica de *Branch & Bound*. Esta técnica puede entenderse como una mejora sobre la búsqueda exhaustiva tradicional, ya que mantiene la optimalidad de fuerza bruta, pero reduce considerablemente la cantidad de soluciones

analizadas gracias al uso de podas.

2.2.1. Explicación del problema

Se tiene un conjunto de productos donde cada producto i está representado por la tupla (s_i, v_i) , donde s_i representa el tamaño del producto y v_i el valor obtenido por enviarlo. Además, se tiene k contenedores iguales, cada uno con capacidad máxima C , y un factor de penalización a aplicado sobre el espacio libre de los contenedores que hayan sido utilizados. Cada producto puede colocarse en un único contenedor o no enviarse. A su vez, la suma de los tamaños de los productos que se encuentran dentro de un mismo contenedor no puede superar su capacidad máxima.

El objetivo consiste en maximizar la ganancia total obtenida por el envío de productos, teniendo en cuenta además la penalización por espacio desaprovechado. Esto está dado por:

$$G = \sum v_i - a \cdot \sum \text{espacio_libre}$$

2.2.2. Búsqueda exhaustiva sin poda

Una solución al problema consiste en realizar una búsqueda exhaustiva de todas las configuraciones posibles. Para cada producto existen $k+1$ decisiones posibles, ya que este puede colocarse en cualquiera de los k contenedores disponibles o directamente no enviarse. Por lo tanto, si existen n productos, el número total de combinaciones posibles que podrían generarse es:

$$(k + 1)^n$$

Esta manera garantiza encontrar la solución óptima ya que analiza todas las combinaciones válidas posibles. Sin embargo, es ineficiente debido a que incluso las ramas que claramente no superan la mejor solución encontrada continúan explorándose hasta el final. Para evitar este problema, se agrega una función límite que permite estimar si una rama todavía tiene posibilidades de mejorar la solución óptima actual y, en caso contrario, descartarla antes de recorrerla completamente.

2.2.3. Propuesta

Se propone construir un árbol de decisiones donde cada nivel representa un producto distinto. Para cada producto i se generan hasta $k + 1$ ramas

posibles, correspondientes a colocarlo en alguno de los k contenedores disponibles o directamente no enviarlo. De esta manera, cada nodo del árbol mantiene el estado parcial de la solución construida hasta el momento.

Durante el recorrido del árbol, el algoritmo calcula una estimación del máximo beneficio que todavía podría alcanzarse usando los productos restantes. Si esta no supera la mejor solución encontrada hasta el momento, entonces la rama actual se poda y deja de explorarse, evitando así recorrer nodos que no conducen a una solución óptima.

2.2.4. Explicación del funcionamiento

El algoritmo analiza los productos uno por uno y, para cada uno de ellos, intenta colocarlo en todos los contenedores donde exista espacio suficiente. Después de cada asignación posible, se actualiza la ganancia parcial obtenida y se calcula la función límite de la nueva rama generada. La exploración continúa solamente si la rama todavía tiene posibilidades de superar la mejor solución encontrada hasta ese momento. Además, también se considera la posibilidad de no enviar el producto actual, generando así otra rama dentro del árbol de búsqueda.

Cuando se alcanza una hoja del árbol, es decir, cuando ya se tomaron decisiones para todos los productos, se calcula la penalización total asociada al espacio libre de los contenedores utilizados y se obtiene la ganancia final de la configuración construida. Finalmente, si esta ganancia resulta mayor que la mejor solución conocida, la solución se actualiza.

2.2.5. Función costo

Representa la ganancia obtenida para un estado dado.

$$G = \sum v_i - a \cdot \sum (C - U_j)$$

Donde U_j es el espacio utilizado en el contenedor “ j ”

2.2.6. Función límite

Permite estimar el máximo beneficio posible que podría alcanzarse.

$$cota_superior = ganancia_actual + \sum valores_restantes$$

Esta función representa una estimación optimista, ya que considera que todos los productos restantes podrían agregarse sin restricciones de capacidad.

$$cota_superior \leq mejor_solucion$$

2.2.7. Complejidades

Para cada producto existen $k + 1$ decisiones posibles, por lo tanto, en el peor caso, el algoritmo puede recorrer:

$$(k + 1)^n$$

Dando una complejidad temporal:

$$T(n) = O((k + 1)^n)$$

Teniendo en cuenta que la profundidad máxima del árbol es igual a la cantidad de productos n , y que las estructuras auxiliares que usamos dependen de la cantidad de contenedores k , la complejidad espacial total del algoritmo queda dada por:

$$E(n) = O(n + k)$$

2.2.8. Ejemplo de ejecución

Teniendo el siguiente conjunto de productos:

```
productos = [(2,10), (5,10), (7,20)]  
k = 2  
C = 8  
a = 5
```

Asignamos los productos 0 y 1 al primer contenedor, mientras que el producto 2 lo colocamos en el segundo contenedor:

```
Contenedor 1 = [0,1]  
Contenedor 2 = [2]
```

De esta manera, los productos del primer contenedor ocupan $2 + 5 = 7$, mientras que la del segundo contenedor 7. Por lo tanto, ambos contenedores tienen 1 unidad de espacio libre.

La penalización total aplicada sobre el espacio no usado es:

$$5(1 + 1) = 10$$

Por otro lado, el valor total obtenido por enviar los productos es:

$$10 + 10 + 20 = 40$$

Finalmente, la ganancia total es de:

$$40 - 10 = 30$$

Por lo tanto, el algoritmo retorna:

$$(30, [[0, 1], [2]])$$

2.2.9. Pseudocódigo

```
main(productos, k, C, a):  
  
    mejor_ganancia = -infinito  
    mejor_solucion = []  
    contenedores = arreglo de k listas vacias  
    ocupacion = arreglo de k ceros  
  
    branch_and_bound(i = 0, productos, contenedores,  
                     ocupacion, ganancia_actual = 0)  
  
    retornar mejor_ganancia, mejor_solucion
```

```
branch_and_bound(i, productos, contenedores, ocupacion,  
ganancia_actual):  
  
    if(i == cantidad_productos):  
  
        calcular penalizacion total  
  
        ganancia_final = ganancia_actual - penalizacion  
  
        if(ganancia_final > mejor_ganancia):
```



```

        actualizar mejor_ganancia
        guardar copia de contenedores

    retornar

cota = ganancia_actual + suma_valores_restantes(i)

if(cota <= mejor_ganancia):

    podar rama
    retornar

cantidad, valor = productos[i]

para cada contenedor j:

    if(ocupacion[j] + cantidad <= C):

        agregar producto i al contenedor j

        ocupacion[j] += cantidad

        branch_and_bound(i + 1, productos, contenedores,
                           ocupacion, ganancia_actual +
valor)

        deshacer cambios

    explorar no enviar el producto actual

```

Para este problema se utilizó la técnica de *Branch & Bound* para resolverlo. Se partió de una solución de búsqueda exhaustiva tradicional que garantiza encontrar el óptimo pero requiere recorrer todas las configuraciones posibles. Sobre esta idea se construyó una mejora mediante el uso de funciones límite y podas, permitiendo descartar ramas que no pueden superar la mejor solución encontrada.

2.3. Los centros de atención

2.3.1. Reducción polinomial a Set Cover

Set Cover es un problema que nos plantea que teniendo un universo de elementos para cubrir y una colección de subconjuntos del mismo, se busca

elegir la menor cantidad de subconjuntos posibles para cubrir todos los elementos del universo. Cómo problema de decisión sería: ¿Existe un conjunto de subconjuntos de tamaño menor o igual a k con el que puedo cubrir todos los elementos?

La clave para poder transformar este problema en uno equivalente a Set Cover es que instalar un centro equivale a elegir un subconjunto que cubre zonas. Y cada zona es un elemento del universo. Dicho esto, tomamos Set Cover en su versión de decisión: Dado un universo U , una colección de subconjuntos $S = \{S_1, S_2, \dots, S_m\}$ tal que $S_j \subseteq U$, y un entero k , ¿existen a lo sumo k subconjuntos cuya unión cubra todo U ?

Queremos demostrar el problema de Centros de Atención (de ahora en más centros), es NP-Hard. Para demostrar que un problema es NP-Hard, tenemos que mostrar que un problema que ya sabemos difícil (NP-Hard) puede transformarse en él, entonces debemos reducir Set Cover al problema de centros de atención, es decir:

$$SetCover \leq_p CentrosdeAtencion$$

Vemos como es una instancia de Set Cover:

$$U = \{u_1, u_2, \dots, u_n\}, S = \{S_1, S_2, \dots, S_m\} \text{ y un límite: } k$$

Construcción de la instancia de centros: A partir de esa instancia armamos una instancia del problema de centros. Por cada elemento $U_i \in U$, creamos una zona $Z_i \in Z$. Por cada subconjunto S_j , creamos un centro posible $F_j \in F$. El centro F_j puede atender a la zona Z_i si y solo si: $U_i \in S_j$

- A cada centro le ponemos costo: $C_j = 1$
- El presupuesto máximo será: $B = k$

Entonces, elegir centros equivale a elegir subconjuntos.

Demostración de equivalencia: Tenemos que demostrar el ida y vuelta. Si Set Cover tiene solución Supongamos que existen a lo sumo k subconjuntos que cubren todo U . Es decir, existe una selección: $\{Sa_1, Sa_2, \dots, Sa_t\}$ con $t \leq k$, tal que: $Sa_1 \cup Sa_2 \cup \dots \cup Sa_t = U$

Entonces instalamos los centros correspondientes: $\{Fa_1, Fa_2, \dots, Fa_t\}$ Como esos subconjuntos cubren todos los elementos de U , los centros correspondientes pueden atender todas las zonas de Z . Además, como cada centro cuesta 1:

$$CostoTotal = t \leq k = B$$

Por lo tanto, la instancia de centros tiene solución. Si la instancia de centros tiene solución Supongamos ahora que existe una selección de centros instalados con costo total menor o igual a $B = k$, y todas las zonas quedan atendidas. Como cada centro cuesta 1, instalar centros con costo total $\leq k$ implica que se instalaron a lo sumo k centros. Tomamos los subconjuntos correspondientes a esos centros. Como todas las zonas son atendidas, eso significa que para cada zona Z_i , hay algún centro F_j instalado que puede atenderla. Por construcción, eso ocurre si y solo si: $U_i \in S_j$

Entonces los subconjuntos elegidos cubren todos los elementos de U . Por lo tanto, existe un Set Cover de tamaño a lo sumo k .

Conclusión: Como Set Cover se reduce polinomialmente al problema de centros de atención, y Set Cover es NP-Completo, entonces el problema de optimización de centros es NP-Hard. La transformación es polinomial porque solamente creamos: Una zona por cada elemento del universo, Un centro por cada subconjunto, Una relación de atención por cada pertenencia $U_i \in S_j$

2.4. Versión de decisión

Para convertirlo en decisión, hay que agregar un presupuesto B y preguntar si existe una solución que no lo supere. Quedaría:

“Dado un conjunto de posibles centros F , un conjunto de zonas Z , costos de instalación C_j , restricciones de qué centros pueden atender a cada zona, y un presupuesto máximo B , ¿existe una selección de centros a instalar y una asignación de zonas a centros instalados tal que todas las zonas sean atendidas y el costo total de instalación sea menor o igual a B ?”

Para demostrar que pertenece a NP, necesitamos ver que, si alguien nos da una solución candidata, podemos verificarla en tiempo polinomial. El certificado podría ser:

```
Certificador (centros, zonas, B, evidencia):
    Si en la evidencia hay algun centro o zona que no
    este en centros o zonas de la instancia:
        Devuelve Falso
    Si en la evidencia la suma de los costos es mayor a B
        Devuelve falso
```

Si en la evidencia no estan todas las zonas asignadas a algun centro: Devuelve Falso Si en la evidencia no estan todos los centros con zonas asignadas instalados: Devuelve falso Devuelve verdadero

De esta forma, verificamos que están todos los centros son de la instancia del problema, que cubren todas las zonas, que están instalados y que los costos no superan a B. Todo eso se puede verificar recorriendo zonas, centros y asignaciones en tiempo polinomial. Por lo tanto, la versión de decisión pertenece a NP.

Para demostrar que un problema es NP-Completo necesitamos probar dos cosas:

1. Que pertenece a NP.
2. Que es NP-Hard.

Ya vimos este problema que pertenece a NP-Hard ya que:

$$SetCover \leq_p CentrosdeAtencion$$

Como Set Cover es NP-Completo, eso demuestra que Centros de Atención es NP-Hard. Entonces:

$$CentrosdeAtencion \in NP \wedge CentrosdeAtencion \in NP - H$$

$$CentrosdeAtencion \in NP - C$$

2.4.1. Otras cuestiones respecto a las clases de complejidad

Si alguien demuestra tener un algoritmo polinomial que resuelve la versión de optimización, entonces también podríamos resolver la versión de decisión en tiempo polinomial. Porque para cualquier algoritmo que pueda resolver un problema de optimización en tiempo polinomial, la verificación de este también se puede hacer en tiempo polinomial.

Nos dicen:

$$X \leq_p CentrosdeAtencion$$

Eso significa que X es a lo sumo tan difícil como Centros de Atención. Como Centros de Atención es NP-Completo, podemos decir que X se puede resolver usando una transformación polinomial hacia un problema NP-Completo. Sin embargo, eso solo no alcanza para decir que X es NP-Hard ni NP-Completo. Para demostrar que X es NP-Hard necesitaríamos lo contrario, es decir:

$CentrosdeAtencion \leq_p X$ o que algún problema NP-Completo se reduzca a X . Lo que sí podemos decir es: Si X es un problema de decisión, entonces $X \in NP$, porque se reduce a un problema que está en NP. No podemos concluir que X sea NP-Completo. No podemos concluir que X sea NP-Hard. Solo sabemos que X no es más difícil que Centros de Atención bajo reducciones polinomiales. Solo sabemos que X es como mucho tan difícil como Centros; no necesariamente NP-Hard ni NP-Completo.

3. Correcciones en la segunda versión

En la presente reentrega del trabajo práctico se han incorporado diversas modificaciones con el fin de subsanar las observaciones señaladas en la corrección inicial. Principalmente, en el apartado de Redes de Flujo, se reformuló la propuesta de solución de la Sección 2.1.2 para modelar la red considerando la bidireccionalidad intrínseca de las relaciones, lo cual conllevó la consecuente actualización del pseudocódigo para garantizar la viabilidad algorítmica.

Asimismo, se robusteció el análisis teórico profundizando la evaluación de la complejidad espacial de las estructuras de datos utilizadas. Por último, se reelaboró integralmente el ejemplo gráfico de la Sección 2.1.4: se detalló el paso a paso de la saturación de la red mediante la búsqueda de flujo máximo y se corrigió la construcción del grafo residual, reflejando de manera precisa la actualización de capacidades directas e inversas de forma previa a la determinación del corte mínimo.

Respecto a la tercer sección, se realizó al completo, retirando el aviso de que se realizaría en esta versión del informe.

4. Conclusión

A partir de este trabajo de investigación y de la puesta en práctica de los conocimientos obtenidos en esta etapa avanzada de la Teoría de Algoritmos,

se ha logrado profundizar en el análisis de problemas de optimización de alta complejidad. El estudio de las Redes de Flujo permitió comprender la modelización de sistemas con restricciones de capacidad, mientras que el abordaje de la Fuerza Bruta y las Clases de Complejidad brindó una perspectiva integral sobre los límites de la eficiencia computacional y la distinción entre problemas tratables e intratables.

Asimismo, se enriqueció la labor realizada mediante una investigación exhaustiva que integró tanto el material bibliográfico de la cátedra como el análisis de publicaciones técnicas y recursos académicos complementarios. Esta base teórica fue fundamental para entender no solo el funcionamiento de los algoritmos, sino también las implicancias de las reducciones entre problemas y la jerarquía de complejidad que rige a la informática moderna.

Finalmente, la implementación en el lenguaje Python de las soluciones propuestas para redes de flujo y algoritmos de búsqueda exhaustiva resultó clave para validar los modelos teóricos. Este proceso permitió observar el comportamiento real de los algoritmos frente a diferentes volúmenes de datos y casos de prueba, consolidando la transición entre el diseño conceptual y la aplicación práctica de soluciones óptimas en entornos de ingeniería.

5. Referencias

1. J. Kleinberg, E. Tardos, Algorithm Design, Addison Wesley (2006). [KT]
2. T. Cormen, C. Leiserson, R. Rivest, C. Stein, Introduction to Algorithms (cuarta edición), MIT Press (2022). [CLRS]
3. Victor Podberesky. Videos de Youtube del Canal vpode. [VPOD]